



Beata Zalewa

Jak zbudować pierwszą
aplikację ASP.NET Core Web API
przy wykorzystaniu
Visual Studio (Code)



Azuronet
Online Azure Meetup

10.08.2021

Godz. 18:30

Jak zbudować pierwszą aplikację ASP.NET Core Web API przy wykorzystaniu Visual Studio (Code)

Beata Zalewa

O mnie



W życiu zawodowym konsultant, Microsoft Certified Trainer, administrator, programista, architekt, od wielu lat freelancer.



Visual C#, SQL Server, SharePoint, Azure



Prywatne życie w liczbach: 1 mąż, 1 córka, 1 kot i 2 psy. Moje hobby to kryminały, praca w ogrodzie i pasieka.



Email: info@zalnet.pl



Linkedin: <https://linkedin.com/in/beatazalewa/>



WWW: <https://www.zalnet.pl/>



Agenda

- Różnica pomiędzy MVC a Web API
- BaseController i atrybut ApiController
- Jak zwracać poprawny status HTTP w odpowiedzi API
- Stworzenie CRUD w API
- Dokumentacja API - dodanie Swagger'a



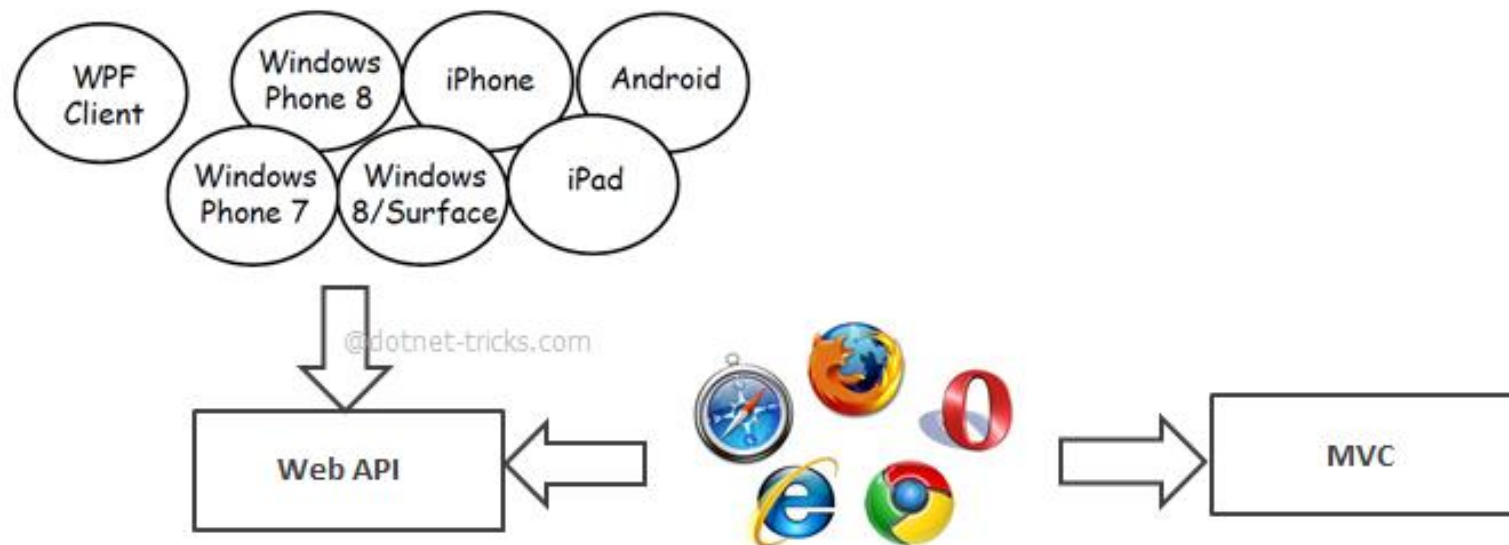
/beatazalewa

Różnica pomiędzy MVC a Web API

- ASP.NET MVC służy do tworzenia aplikacji internetowych, które zwracają zarówno widoki, jak i dane.
- Interfejs API sieci Web Asp.Net służy do tworzenia pełnowartościowych usług HTTP w łatwy i prosty sposób, który zwraca tylko dane, a nie widok.
- W skrócie mówiąc w kontrolerze MVC zwracamy widoki (prezentujemy dane), a w kontrolerze API udostępniamy.

Różnica pomiędzy MVC a Web API

Źródło: *dotnet-tricks.com*



Budowanie kontrolerów dla zasobów REST

- REST oznacza wzorzec przeniesienia stanu reprezentacyjnego, który zawiera 2 elementy:
- Metody akcji (**action methods**), które wykonują określone operacje, a następnie dostarczają klientowi pewne dane. Te metody są ozdobione atrybutami, które sprawiają, że są wywoływane tylko przez żądania **HTTP** (np. [HttpGet], [HttpPost]).
- Adresy URL określające zadania operacyjne. Operacjami tymi mogą być – przesłanie całości lub części danych, dodanie, usunięcie lub aktualizacja rekordów.



/beatazalewa

ControllerBase i atrybut ApiController

- Kontrolery API mają 3 cechy:
 1. Powinny mieć atrybut [ApiController]. Ten atrybut informuje, że kontroler będzie obsługiwał odpowiedzi HTTP API.
 2. Powinny pochodzić z klasy ControllerBase, a nie z klasy Controller.
 3. Powinny mieć na nich zastosowane routing atrybutów, takie jak [Route("someUrl/[controller]")]. Ta będzie bardzo podobna do struktury samego żądania.
- Główną różnicę stanowi pierwsza linijka.
- Zamiast metody HTTP i ścieżki do zasobu otrzymamy odpowiedni kod odpowiedzi z serwera.

ControllerBase i atrybut ApiController

```
[ApiController]
```

```
[Route("someURL/[controller]")]
```

```
public class ExampleController :  
    ControllerBase
```

- Powodem, dla którego nie należy tworzyć kontrolera Web API przez dziedziczenie z klasy Controller, jest to, że klasa Controller pochodzi z klasy ControllerBase i dodaje obsługę widoków, więc służy do obsługi widoków, a nie żądań Web API. Istnieje wyjątek od tej reguły: jeśli planujemy używać tego samego kontrolera zarówno dla widoków, jak i WebAPI, dziedziczymy go tylko z klasy Controller.

Jak zwracać poprawny status HTTP w odpowiedzi API

Klasa **ControllerBase** udostępnia wiele właściwości i metod, które są przydatne do obsługi żądań HTTP.

Niektóre z nich to:

Nazwa	Opis
BadRequest	Zwraca kod statusu 400.
Ok	Zwraca kod statusu 200 status z obiektem result.
NotFound	Zwraca kod statusu 404.
PhysicalFile	Zwraca <u>plik</u> .



/beatazalewa

Jak zwracać poprawny status HTTP w odpowiedzi API

```
[HttpGet("{id}")]  
  
public ActionResult<Reservation> Get(int id)  
{  
    if (id == 0)  
        return BadRequest("Value must be passed  
in the request body.");  
    return Ok(repository[id]);  
}
```

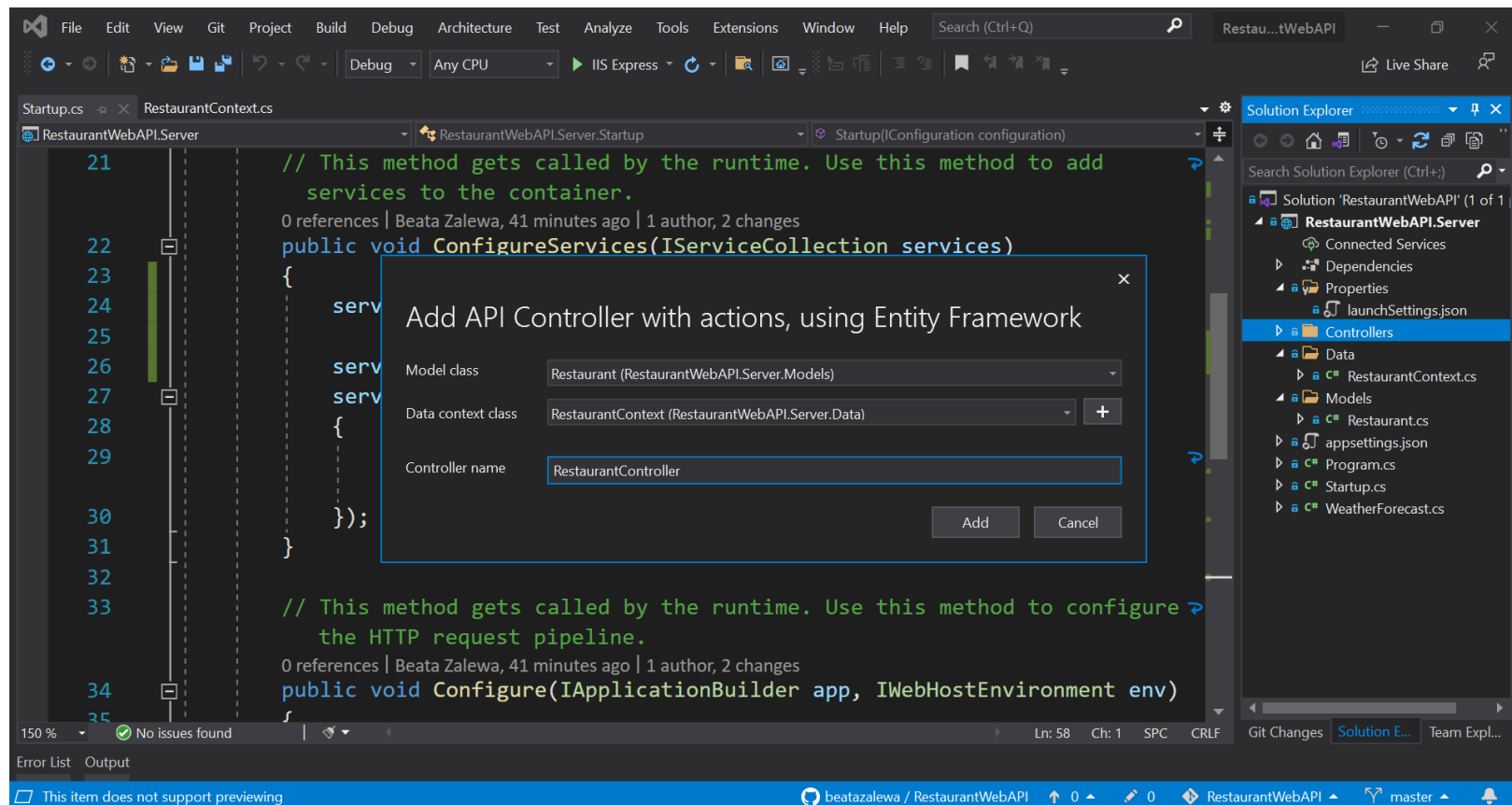
Routing dla API

- Routing Web API jest podobny do routingu ASP.NET MVC.
- Kieruje przychodzące żądanie HTTP do określonej metody akcji na kontrolerze API.

Web API obsługuje dwa typy routingu:

- Routing oparty na konwencji (Convention-based Routing)
- Routing atrybutów (Attribute Routing)

Stworzenie CRUD w API





```
[{"id":1,"name":"Beata","startLocation":"New York","endLocation":"Warsaw"}, {"id":2,"name":"Marcin","startLocation":"New Jersey","endLocation":"Boston"}, {"id":3,"name":"Luis","startLocation":"London","endLocation":"Paris"}]
```

Formatowanie
odpowiedzi - json, xml

Formatowanie odpowiedzi - json, xml

Możemy to zmienić i sprawić, by WebAPI zwracało dane w formacie XML, a nie JSON.

W tym celu należy dodać metodę

AddXmlDataContractSerializerFormatters() w metodzie **ConfigureServices** w klasie **Startup.cs**.

```
services.AddControllersWithViews().AddNewtonsoftJs  
on()  
  
.AddXmlDataContractSerializerFormatters();
```

Formatowanie odpowiedzi - json, xml

Możemy to zmienić i sprawić, by WebAPI zwracało dane w formacie XML, a nie JSON.

W tym celu należy dodać metodę

AddXmlDataContractSerializerFormatters() w metodzie **ConfigureServices** w klasie **Startup.cs**.

```
services.AddControllersWithViews().AddNewtonsoftJson().AddXmlDataContractSerializerFormatters();
```



The screenshot shows a web browser window with the address bar displaying `https://localhost:44302/api/Reservation`. Below the address bar, a message states: "This XML file does not appear to have any style information associated with it. The document tree is shown below." The XML document tree is displayed as follows:

```
<ArrayOfReservation xmlns:i="http://www.w3.org/2001/XMLSchema-instance" xmlns="http://schemas.datacontract.org/2004/07/APIControllers.Models">
  <Reservation>
    <EndLocation>Warsaw</EndLocation>
    <Id>1</Id>
    <Name>Beata</Name>
    <StartLocation>New York</StartLocation>
  </Reservation>
  <Reservation>
    <EndLocation>Boston</EndLocation>
    <Id>2</Id>
    <Name>Marcin</Name>
    <StartLocation>New Jersey</StartLocation>
  </Reservation>
  <Reservation>
    <EndLocation>Paris</EndLocation>
    <Id>3</Id>
    <Name>Luis</Name>
    <StartLocation>London</StartLocation>
  </Reservation>
</ArrayOfReservation>
```




/beatazalewa

Dokumentacja API - dodanie Swagger'a

- RoutingSwagger służy do generowania przydatnej dokumentacji i stron pomocy dla Web API.
- Ale Swagger to nie tylko biblioteka do generowania dokumentacji tworzonego api.
- To przede wszystkim cały ekosystem narzędzi, które umożliwiają nam tworzenie RESTowego api.
- Ekosystem zawiera narzędzia do projektowania api, jego dokumentowania czy też testowania.
- Otrzymujemy dziesiątki narzędzi, które wspierają różne języki programowania, technologie, frameworki. Strona główna Swaggera – <https://swagger.io/>.
- Wskazówka: Więcej informacji na temat Swaggera: dokumentacja ASP.NET Core Web API ze Swaggerem

[ASP.NET Core web API documentation with Swagger / OpenAPI | Microsoft Docs](#)

Dokumentacja API - dodanie Swagger'a



Swagger
Supported by SMARTBEAR

Select a definition RestaurantWebApi v1

RestaurantWebApi v1 OAS3

</swagger/v1/swagger.json>

WeatherForecast

GET /WeatherForecast

Parameters

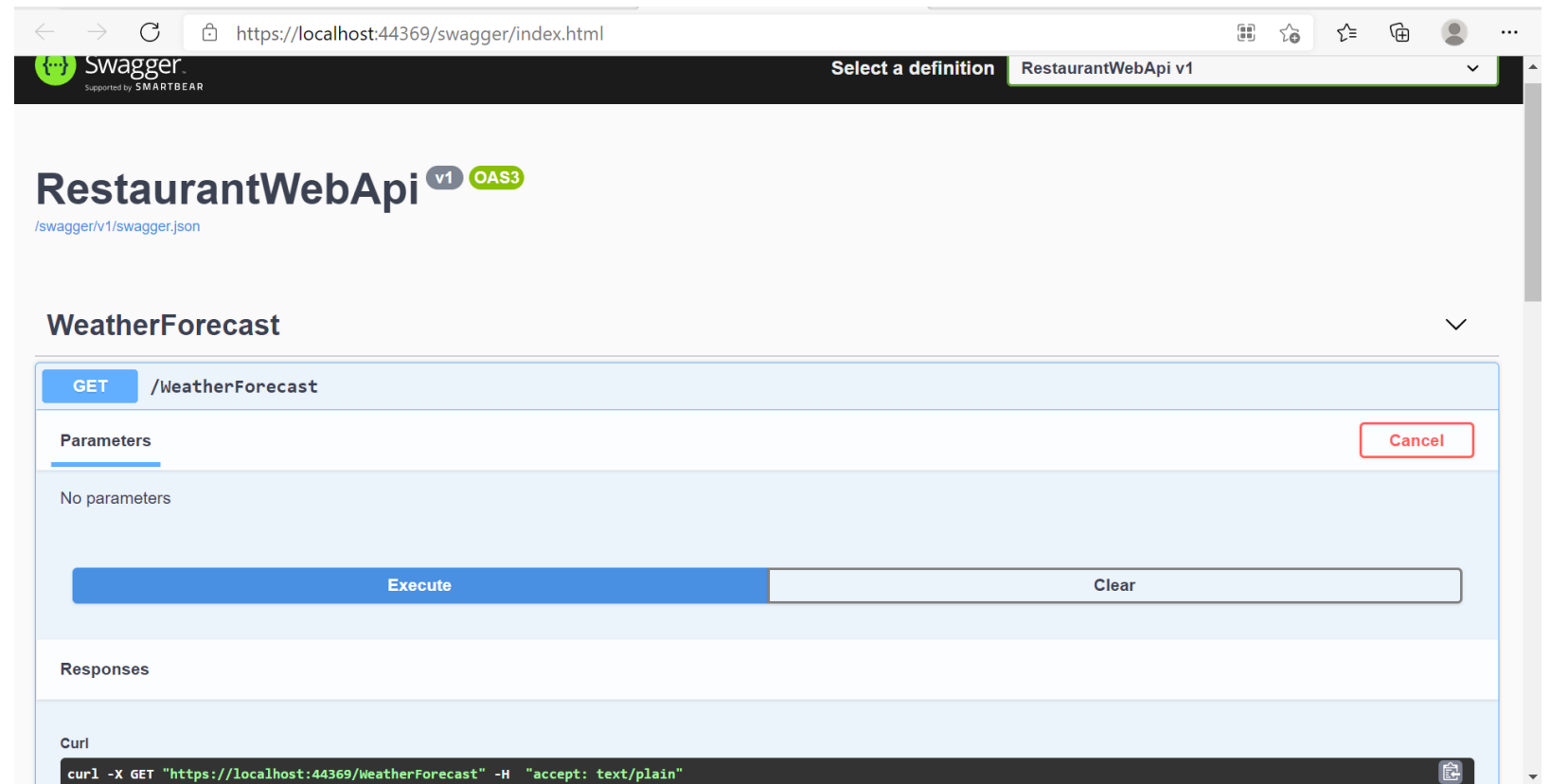
No parameters

Responses

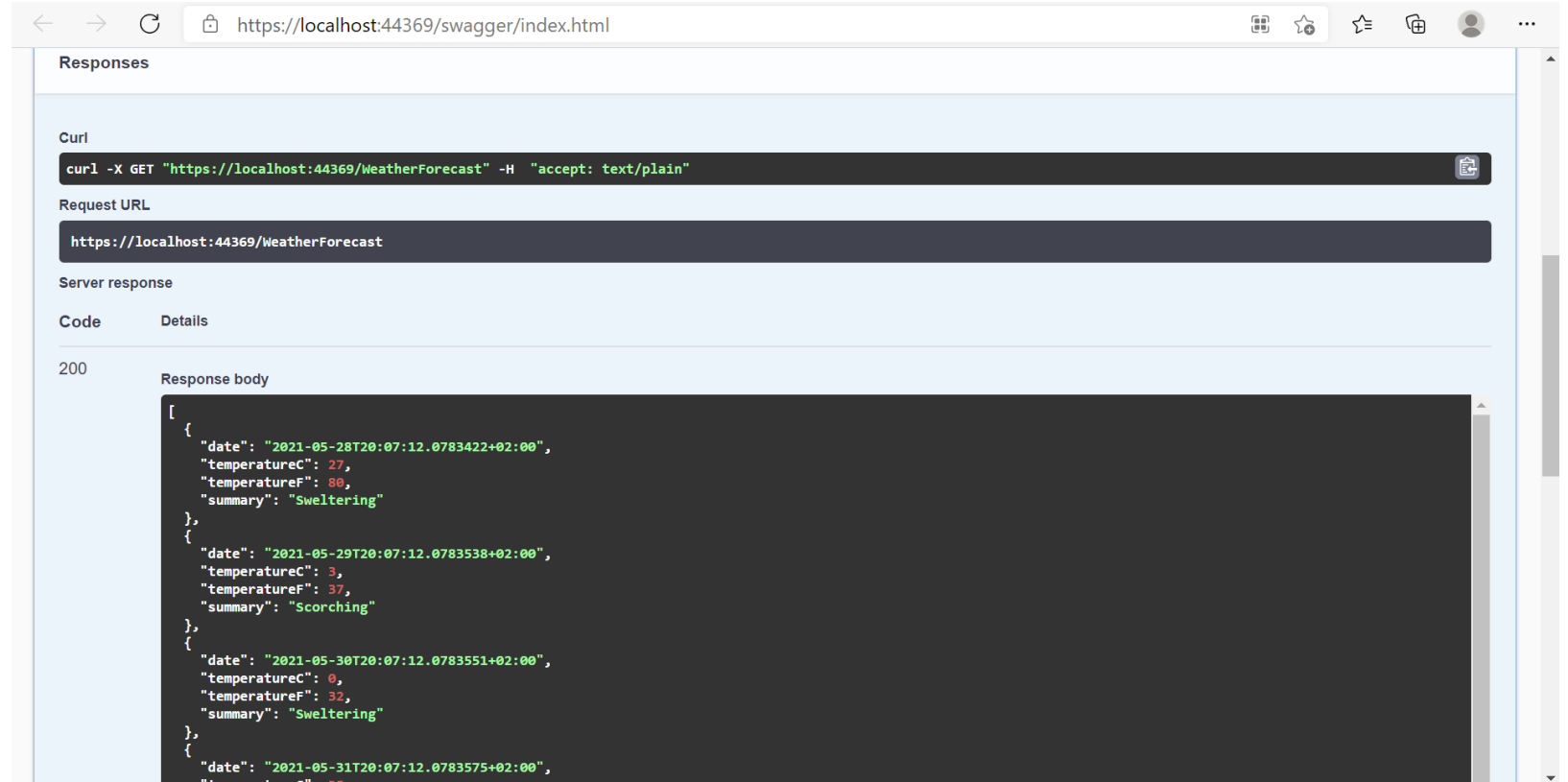
Code	Description	Links
200	Success	No links

Try it out

Dokumentacja API - dodanie Swagger'a



Dokumentacja API - dodanie Swagger'a



Demo

Stop teorii, czas na praktykę !



/beatazalewa



/beatazalewa

API	Description	Request body	Response body
GET /api/Restaurant	Get all restaurants	None	Array of restaurants
GET /api/ Restaurant /{id}	Get a restaurant by ID	None	Restaurant
POST /api/ Restaurant	Add a new restaurant	Restaurant	Restaurant
PUT /api/ Restaurant /{id}	Update an existing restaurant	Restaurant	None
DELETE /api/ Restaurant /{id}	Delete a restaurant	None	None

Demo

Q & A

Email: info@zalnet.pl

Kod aplikacji:

<https://github.com/beatazalewa/RestaurantWebAPI>

Tutorial do aplikacji z demo:

<https://zalnet.pl/blog>

Dziękuję za poświęcony czas 😊